<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN"> <html><head> <title>404 Not Found</title> </head><body> <h1>Not Found</h1> The requested URL was not found on this server. </body></html>

13

Test-Time Compute: Scaling Inference

The Test-Time Compute Idea

Every chapter so far has focused on making models better during *training*: supervised fine-tuning teaches basic competence, RLHF and DPO align with preferences, GRPO discovers reasoning strategies. Once training is done, the model is fixed. At inference time, it generates one response and returns it. The quality of that response is entirely determined by what the model learned during training.

Test-time compute flips this assumption. Instead of generating one response, you spend extra computation *at inference time* to produce a better answer. Generate multiple candidates and pick the best one. Have the model critique and revise its own output. Explore multiple reasoning paths in parallel and select the most promising one. The model's weights do not change—you are trading compute for quality on a per-query basis.

This is the idea behind OpenAI's o1 (Chapter 12), and it applies broadly to any model trained with any method from this book. A model trained with GRPO

can be made even better at inference time by generating multiple solutions and selecting the best one with a PRM (Chapter 11). A model trained with DPO can be improved by self-correction loops. The techniques in this chapter are orthogonal to training: they work on top of whatever the model has already learned.

The core trade-off.

Training-time compute is a **one-time investment** that improves every future query. Test-time compute is a **per-query cost** that improves only the current response.

Training is more efficient when you serve many queries (the cost amortizes). Test-time compute is more efficient when queries are rare but high-stakes (each one is worth spending extra on).

Most production systems use both: heavy training to build a strong base model, then selective test-time compute for the hardest or most important queries.

Decoding Fundamentals

Before covering test-time scaling strategies, it helps to review how a language model generates text in the first place. Every strategy in this chapter builds on the basic decoding loop: at each token position, the model produces a probability distribution over the vocabulary, and a *decoding strategy* decides which token to pick.

Greedy decoding always picks the highest-probability token. This is deterministic and fast but can get stuck in repetitive loops and misses good responses that start with a lower-probability token.

Sampling draws randomly from the probability distribution. This produces diverse outputs but can be incoherent if low-probability tokens are selected.

Temperature scaling adjusts the sharpness of the distribution before sampling:

$$P(\text{token}_i) = \frac{\exp(\text{logit}_i / T)}{\sum_j \exp(\text{logit}_j / T)}$$

Low temperature ($T = 0.1$) concentrates probability on the top tokens—nearly deterministic. High temperature ($T = 2.0$) flattens the distribution—more random and diverse. Medium temperature ($T = 0.7$–1.0) is the standard operating range for most tasks.

Top-k sampling restricts the distribution to the k highest-probability tokens, then samples from that truncated set. This prevents the model from selecting extremely unlikely tokens while preserving some diversity. Typical values are $k = 40$–100.

Top-p (nucleus) sampling restricts the distribution to the smallest set of tokens whose cumulative probability exceeds p. Unlike top-k, this adapts automatically: when the model is confident (one token dominates), the set is small; when the model is uncertain, the set is larger. Typical values are $p = 0.9$–0.95.

Temperature for test-time scaling. All the strategies in this chapter—best-of-N, self-consistency, tree search—require generating *multiple diverse* outputs. Greedy decoding produces the same output every time, so it is useless for these methods. You need sampling with moderate temperature ($T = 0.6$–1.0) to get diversity. Too low and all candidates are identical; too high and they are incoherent. The right temperature balances diversity (different reasoning paths) against quality (each path is coherent).

Best-of-N and Rejection Sampling

The simplest test-time scaling strategy is to generate N responses and keep only the best one. This requires a scoring function—a reward model (Chapter 9), a PRM (Chapter 11), a verifier, or even just majority voting.

Best-of-N with a Scorer

Generate N candidates, score each, return the highest-scoring one. The quality improvement comes from sampling: even if the model produces a good response only 30% of the time, generating $N = 10$ candidates gives a $1 - 0.7^{10} = 97\%$ chance that at least one is good.

Best-of-N scaling behavior.

Let p be the probability that a single sample is "good" (above some quality threshold).

Probability that at least one of N samples is good: $P(\text{success}) = 1 - (1 - p)^N$.

p	$N = 1$	$N = 3$	$N = 5$	$N = 10$	$N = 20$
10%	10%	27%	41%	65%	88%
30%	30%	66%	83%	97%	99.9%
50%	50%	88%	97%	99.9%	~100%

Even a weak model ($p = 10\%$) can produce good outputs reliably with enough candidates. The cost is $N\times$ normal inference, so there is a direct quality–cost trade-off.

Choosing the scorer matters enormously. Chapter 11 showed that PRM-scored best-of-100 achieves 72.2% on the MATH benchmark, versus 58.3% for ORM-scored best-of-100, versus 42.0% for greedy decoding. The model is the same in all three cases—the only difference is how candidates are ranked. Investing in a good scorer (especially a PRM for reasoning tasks) amplifies the value of every candidate you generate.

Rejection Sampling

Rejection sampling is a variant of best-of-N with a threshold instead of a ranking. Generate candidates one at a time, score each, and return the first one that exceeds a threshold τ. If no candidate passes within N attempts, return the best one seen so far.

Rejection sampling analysis.

Let $p_{\text{accept}} = P(r \geq \tau)$ be the probability a single sample passes the threshold.

Expected number of attempts: $\mathbb{E}[\text{attempts}] = \frac{1}{p_{\text{accept}}}$.

With $p_{\text{accept}} = 0.4$: expect 2.5 attempts on average. With $N = 5$ max attempts: 92% chance of finding an acceptable response.

The threshold trade-off: Strict threshold (τ high) means higher quality but more rejections—expensive. Lenient threshold (τ low) is faster but accepts mediocre responses. Set τ based on your cost-quality budget and latency constraints.

Rejection sampling is more latency-friendly than full best-of-N because it can return the first good candidate without generating all N. On average, it generates fewer candidates. But it requires a calibrated scorer that produces meaningful absolute scores (not just relative rankings), which is harder to build than a ranker.

Self-Consistency

Self-consistency is best-of-N without a learned scorer. Generate N solutions, extract the final answer from each, and return the answer that appears most frequently. The insight is that correct reasoning paths converge on the same answer, while incorrect paths produce *different* wrong answers.

Example. Problem: "If apples cost $2 each and oranges cost $3 each, and you buy 4 apples and 3 oranges, what is the total?"

Generate 5 solutions. Three arrive at $17 through correct reasoning ($4\times2+3\times3=17$). One gets $35 (incorrectly averaging prices). One gets $17.50 (different error). Majority vote: $17 wins with 3 votes. The correct answer emerges because the correct reasoning path is more robust—there are many ways to get it wrong, but essentially one way to get it right.

When self-consistency works and when it does not.

Works well for: Math problems, multiple-choice questions, logic puzzles, code generation (run tests, majority wins)—any task with a single correct answer that can be extracted and compared.

Does not work for: Creative writing (no single correct answer), open-ended questions, subjective tasks, or problems where the model consistently makes the same error (majority vote amplifies systematic biases).

Self-Correction Loops

Best-of-N generates candidates independently. Self-correction generates candidates *sequentially*, where each attempt is informed by feedback on the previous one. The pattern is: generate a response, critique it, then revise based on the critique.

The Self-Correction Pattern

Step 1: Generate an initial response.

Step 2: Critique. Ask the model (or a separate model) to review the response and identify errors, weaknesses, or improvements.

Step 3: Check. If the critique finds no significant issues, return the response. Otherwise, continue.

Step 4: Revise. Generate an improved version that addresses the critique's feedback.

Step 5: Repeat steps 2–4 up to a maximum number of iterations (typically 2–3).

Total cost: 2–4× normal inference (each iteration requires generation + critique).

When self-correction works. The model must be better at *verification* than *generation*—that is, it must be able to recognize errors it could not avoid making in the first place. This is common in math (checking a solution is easier than finding it), code (running tests is easier than writing bug-free code), and factual questions (looking up an answer is easier than recalling it perfectly).

When self-correction fails. Three failure modes are common. First, the model's critique ability may be poor—it cannot identify real errors, or it hallucinates errors that do not exist, leading to "corrections" that make the response worse. Second, the model may get stuck in loops, fixing one problem while introducing another, oscillating without converging. Third, on tasks where the model is confidently wrong (systematic errors), the critique will agree with the initial response and no correction occurs.

Improving self-correction reliability. Use an external signal (verifier, code execution, retrieval) in the critique step rather than relying solely on the model's self-assessment. A model critiquing its own math is unreliable; a model whose critique step includes running the solution through a symbolic checker is much more reliable. The self-correction loop *becomes* powerful when the *critique* step has access to information the model didn't have during generation.

Tree Search: Beam Search and MCTS

Best-of-N and self-correction operate on *complete* responses. Tree search operates on *partial* responses, making decisions at intermediate points in the generation process. This is more powerful because it can redirect reasoning before errors propagate.

Beam Search

Beam search maintains K parallel "beams" (partial sequences) at each generation step. At each token position, it expands each beam with the top candidates, scores all resulting partial sequences, and keeps only the top K.

Standard beam search scores partial sequences by cumulative log probability. This works well for translation and summarization but does not account for reasoning quality.

Reward-guided beam search replaces log probability with a learned scorer. At each step (or at natural breakpoints like sentence boundaries), a value function or PRM scores each partial sequence, and the top K beams are kept. This focuses computation on the most promising reasoning paths.

The cost is $K\times$ normal inference (maintaining K parallel beams). The benefit is that errors in early reasoning steps can be caught and pruned before they corrupt the rest of the response. A model that makes a wrong assumption in Step 1 will generate a flawed solution no matter how good its subsequent reasoning is. Beam search can prune the wrong-assumption beam early and invest computation in more promising paths.

Monte Carlo Tree Search (MCTS)

MCTS is a more sophisticated search strategy, originally developed for game-playing AI (it powered AlphaGo). Instead of maintaining K parallel beams uniformly, MCTS allocates computation *adaptively*—spending more time exploring promising branches and less time on dead ends.

MCTS for LLM Reasoning: Four Steps

1. Selection. Starting from the root (the prompt), traverse the tree by picking the most promising child at each node, balancing exploitation (high-scoring paths) with exploration (under-explored paths). This uses the UCB formula: $\text{UCB} = \bar{r} + c\sqrt{\frac{\ln N_{\text{parent}}}{N_{\text{child}}}}$.

2. Expansion. At a leaf node, generate possible next steps (e.g., different first reasoning steps for a math problem).

3. Simulation (rollout). For each new child, complete the response (using greedy or sampled decoding) and score the final result with a verifier or reward model.

4. Backpropagation. Update the value estimates for every ancestor node based on the rollout result. Good outcomes increase the value of the path that led to them; bad outcomes decrease it.

Repeat for a compute budget (e.g., 100 rollouts). Return the highest-value complete path.

MCTS walkthrough. Problem: "Solve $2x + 5 = 13$."

The model considers three possible first steps: (A) "Subtract 5 from both sides," (B) "Divide both sides by 2," (C) "Add 5 to both sides."

Round 1: Try each once. Path A → "$2x = 8$" → "$x = 4$" (correct, reward $+1$). Path B → "$x + 2.5 = 6.5$" → "$x = 4$" (correct but more steps, reward $+1$). Path C → "$2x + 10 = 18$" → wrong direction (reward 0). Updated values: A = 1.0, B = 0.8 (penalized for inefficiency), C = 0.0.

Round 2: Focus on promising branches. MCTS explores more variations of path A (all correct). Path A's value stays at 1.0 (high confidence). Path C is abandoned (no further rollouts spent on it).

Final decision: Return the best completion found along path A. MCTS spent 1 rollout on the dead-end path C and multiple rollouts on the promising path A—this adaptive allocation is its key advantage over beam search, which would have maintained all three paths equally.

Beam search vs MCTS.

Beam search is simpler, cheaper per step, and works well when the scorer is reliable at every intermediate point. It is the stan-

dard choice for production systems where latency matters.

MCTS is more powerful but significantly more expensive (each node requires a full rollout to score). It excels on hard reasoning problems where early decisions matter and the search space is large. It is the standard choice for research systems targeting maximum accuracy on benchmarks.

Rule of thumb: If $K = 4$–8 beams are sufficient, use beam search. If you need to explore dozens of reasoning paths on hard problems, use MCTS.

Combining Strategies

Production systems rarely use a single test-time strategy. The most effective approaches combine multiple techniques, with each stage filtering or improving candidates.

Recipe 1: High-Quality Reasoning (research/benchmarks)

Temperature $T = 0.7$. Generate $N = 10$ solutions via sampling. Score each with PRM (Chapter 11). Keep top 3 by PRM score. Take majority vote among top 3.

Cost: ~10× normal inference. Benefit: 25–40% accuracy improvement on hard reasoning tasks. This combines diversity (sampling), quality filtering (PRM), and robustness (majority vote).

Recipe 2: Fast Production System

Temperature $T = 0.8$. Generate $N = 3$ candidates. Score with lightweight reward model. Return highest-scoring candidate.

Cost: 3× normal inference. Benefit: 10–15% quality improvement, low latency. Suitable for high-volume APIs where per-query cost matters.

Recipe 3: Maximum Accuracy (competition math, formal proofs)

Use MCTS with a PRM as the value function. Budget: 100 rollouts per problem. At each reasoning step, expand the 3 most promising branches. Score partial solutions with PRM, complete solutions with verifier. Return the verified-correct solution with highest PRM score.

Cost: 50–100× normal inference. Benefit: can solve problems the base model gets wrong 95% of the time with greedy decoding. Only justified for high-stakes, latency-tolerant applications.

Train-Time vs Test-Time Compute: The Allocation Question

With a fixed total compute budget, how should you split investment between training and inference?

Chapter 12 showed that GRPO training with 10× compute produces a 23-point accuracy gain on MATH (45% → 68%). This section shows that test-time compute can produce comparable gains—but the economics are different.

Training compute scales sublinearly but amortizes across all queries. Doubling training compute does not double model quality. But the improved model serves every future query at no additional cost. For a model serving 100 million queries, even an expensive training investment is cheap per query.

Test-time compute scales well but costs per query. Best-of-10 with PRM scoring can improve accuracy by 15–20 points on reasoning tasks. But every single query costs 10× more. For a high-volume service, this is expensive.

When to invest in training vs inference.

Favor training when: You serve many queries (cost amortizes), you need consistent quality across all queries, and you can afford the upfront investment. Training makes every query better for free.

Favor test-time compute when: Queries are rare but high-value (legal analysis, medical reasoning), difficulty varies (easy queries need $N = 1$, hard queries need $N = 10+$), or you need to deploy quickly and improve quality without retraining.

The adaptive approach: Many production systems use a difficulty classifier to route queries. Easy queries get greedy decoding (cheap). Medium queries get best-of-3 (moderate cost). Hard queries get best-of-10 with PRM scoring or MCTS (expensive). This allocates test-time compute where it has the most impact.

The practical frontier. The best systems combine both. Train hard (GRPO, PPO, DPO—Chapters 6–12) to build the strongest possible base model. Then apply test-time compute selectively on the hardest queries. Chapter 12's DeepSeek-R1 invested heavily in training-time compute (GRPO with curriculum learning). OpenAI's o1 reportedly invests heavily in test-time compute (extended "thinking" at inference). The optimal split depends on your query volume, latency requirements, and the difficulty distribution of your workload.

Chapter 17 integrates training and inference strategies into end-to-end recipes for chatbots, reasoners, and agents.